

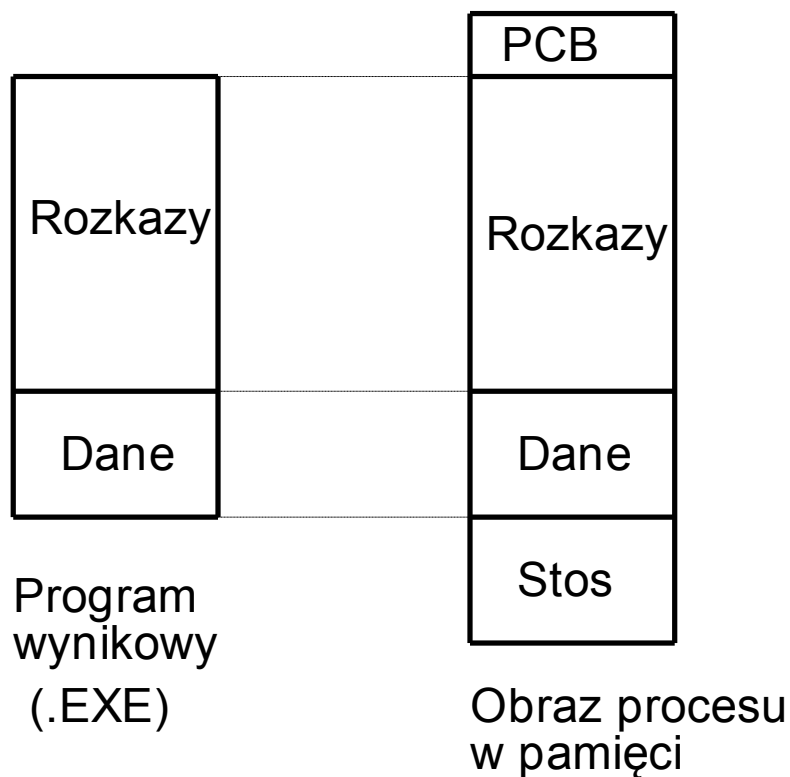
# Oprogramowanie Systemowe (System Software)

Tomasz Dziubich

p. 530

[dziubich@eti.pg.gda.pl](mailto:dziubich@eti.pg.gda.pl)

# Ładowanie programów do pamięci



- ładowanie bezwzględne,
- ładowanie relokowalne,
- ładowanie dynamiczne.

# Pliki linkowalne i wykonywalne

Pliki zawierające kod i dane programu, zakodowane w sposób zrozumiały dla procesora to object files

Plik object może być:

- linkowalny (linkable),
- wykonywalny (executable),
- ładowalny (loadable),
- mogą też występować kombinacje trzech ww. możliwości;

W systemie Windows/DOS pliki linkowalne są kodowane w formacie OMF, a wykonywalne w formacie EXE — oba te formaty są mają odmienną budowę;

# Składowe plików *object*

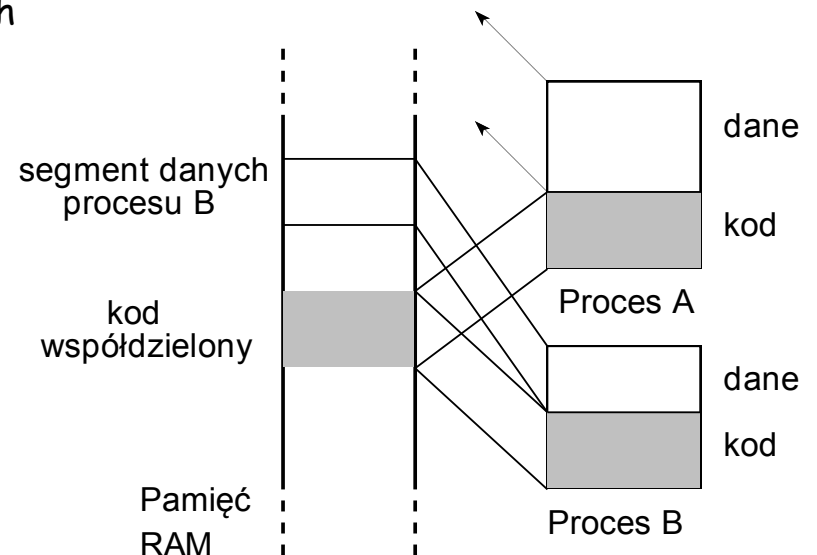
- ✓ *nagłówek* (ang. header) — zawiera ogólne informacje o pliku, data tworzenia, rozmiar kodu, itp.;
- ✓ *kod wykonywalny* (ang. object code) — instrukcje i dane w postaci binarnej, wygenerowane przez kompilator lub assembler;
- ✓ *opis relokacji* — lista miejsc w kodzie wykonywalnym, które muszą być skorygowane;
- ✓ *symbole* — symbole globalne zdefiniowane w danym module, symbole importowane z innych modułów lub symbole definiowane przez linker;
- ✓ *informacje dla debuggera* — dodatkowe informacje potrzebne dla debuggera, jak np. symbole lokalne, numery linii programu źródłowego, opisy struktur danych, itp.

# Format COM

- bezpośrednio przed rozpoczęciem wykonywania programu system operacyjny ładuje plik do wolnego obszaru pamięci
- początkowe 256 bajtów tego obszaru zajmuje blok PCB, tu nazywany *przedrostkiem segmentu programu* (ang. PSP — program segment prefix)
- zawartość pliku wpisywana jest do obszaru począwszy od offsetu 100H
- rejestry segmentowe są ustawiane tak, że wskazują początek bloku PSP, rejestr SP wskazuje koniec segmentu, po czym następuje skok do początku programu;

# Format a.out

```
int a_magic;           // magic number
int a_text;            // rozmiar segmentu kodu (text segment)
int a_data;            // rozmiar segmentu danych inicjalizowanych
                        // (initialized data)
int a_bss;             // rozmiar segmentu danych nieinicjalizowanych
                        // (uninitialized data)
int a_syms;            // rozmiar tablicy symboli (symbol table)
int a_entry;           // punkt wejścia
int a_trsize;          // rozmiar relokacji kodu (text relocation size)
int a_drsize;          // rozmiar tablicy relokacji danych
                        // (data relocation size)
```



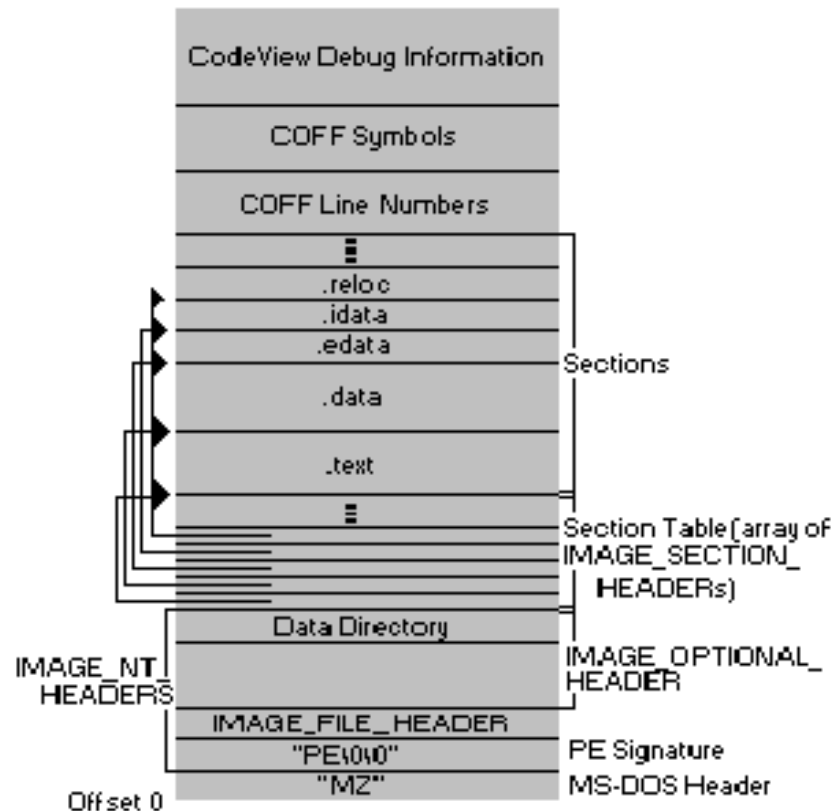
# DOS EXE

```
char signature[2] = "MZ";  
short lastsize;  
short nblocks;  
short nreloc;  
short hdrsize;  
short minalloc;  
short maxalloc;  
  
void far * sp;  
short checksum;  
void far * ip;  
short relocpos;  
  
short noverlay;  
char extra[ ];  
void far * relocs[ ];
```

```
// identyfikator formatu (magic number)  
// długość pliku mod 512  
// liczba bloków 512 zajmowanych przez plik  
// liczba pozycji w tablicy modyfikacji obrazu programu  
// rozmiar nagłówek w (16-bajtowych) paragrafach  
// minimalna liczba przydzielanych dodatkowych paragrafów  
// maksymalna liczba przydzielanych dodatkowych  
// paragrafów  
// zawartość początkowa rejestrów SS:SP  
// suma kontrolna  
// zawartość początkowa rejestrów CS:IP  
// położenie tablicy modyfikacji obrazu  
// programu (ang. location of relocation  
// fixup table)  
// liczba nakładek programu  
// dodatkowy obszar dla nakładek, itp.  
// kolejne pozycje tablicy relokacji
```

# Format PE

- DOS Stub + Sygnatura
  - Sygnatura PE
  - Wykonywalny program DOS
- IMAGE\_NT\_SIGNATURE (0x00004550)
- Nagłówek pliku (COFF)
- Opcjonalny nagłówek (PE Header)
- Katalogi danych
  - Ulokowane w statycznych offsetach
  - Wskazują na specyficzne struktury
    - Imports, Exports, etc
- Nagłówki sekcji
- Sekcje





# Format PE

```
char signature[4] = "PE\0\0"; // identyfikator (magic number)
                                // określa także porządek bajtów

// nagłówek formatu COFF
unsigned short Machine;        // typ procesora:
                                // 0x14C for 80386, itd.

unsigned short NumberOfSections; // liczba sekcji
unsigned long  TimeDateStamp;    // czas utworzenia lub zero
unsigned long  PointerToSymbolTable; // offset tablicy
                                // symboli (COFF) lub zero

unsigned long  NumberOfSymbols;  // liczba pozycji
                                // w tablicy symboli

unsigned short SizeOfOptionalHeader; // rozmiar
                                // nagłówka opcjonalnego

unsigned short Characteristics; // 02 = executable,
                                // 0x200=nonrelocatable,
                                // 0x2000 = DLL
```

# Opcjonalny nagłówek PE

```
typedef struct _OPTHEADERS{
    WORD    Magic;
    BYTE    MajorLinkerVersion;
    BYTE    MinorLinkerVersion;
    DWORD    SizeOfCode;                // code segment size
    DWORD    SizeOfInitializedData;    // data segment size
    DWORD    SizeOfUninitializedData;  // data segment size
    DWORD    AddressOfEntryPoint;      // entry point
    DWORD    BaseOfCode;
    DWORD    BaseOfData;
    DWORD    ImageBase;
    DWORD    SectionAlignment;
    DWORD    FileAlignment;
    WORD     MajorOperatingSystemVersion;
    WORD     MinorOperatingSystemVersion;
    WORD     MajorSubsystemVersion;
    WORD     MinorSubsystemVersion;
    DWORD    Reserved;
    DWORD    SizeOfImage;
    DWORD    SizeOfHeaders;
    DWORD    CheckSum;
    WORD     Subsystem;
    DWORD    DllCharacteristics;
    DWORD    SizeOfStackReserve;
    DWORD    SizeOfStackCommit;
    DWORD    SizeOfHeapReserve;
    DWORD    SizeOfHeapCommit;
    DWORD    LoaderFlags;
    DWORD    NumberOfRvaAndSizes;      // data directories
} OPTHEADERS, *POPTHEADERS;
```

# Format PE

- Wpisy sekcji
  - Wyliczanie Adresu:

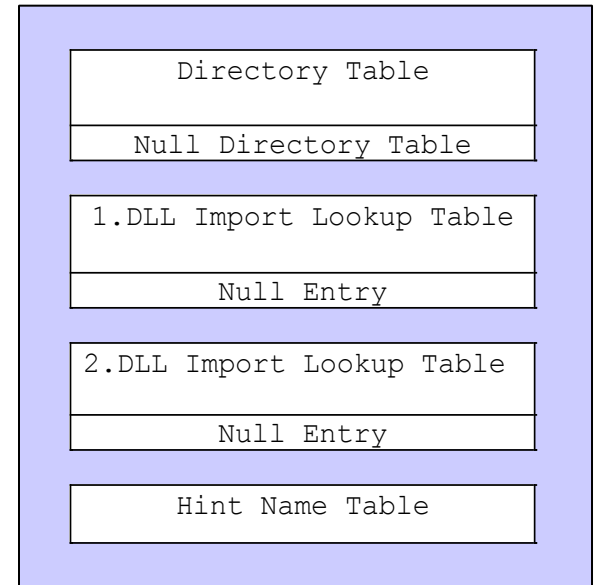
```
base_addr + *(uint32)(base_addr + 0x3c)
+ sizeof(COFF) + PCOFF->SizeOfOptionalHeader
```

- Wpisy z relokacjami są obecne tylko w pliku object

```
typedef struct _SECTIONTABLES {
    BYTE    Name[8];                // Section name
    DWORD   VirtualSize;            // Size of section in memory
    DWORD   VirtualAddress;        // Address of mapped section
    DWORD   SizeOfRawData;         // Size of section on disk
    DWORD   PointerToRawData;      // Section file offset
    DWORD   PointerToRelocations;  // Relocation entries file offset
    DWORD   PointerToLineNumbers;  // Line-number entries file offset
    WORD    NumberOfRelocations;   // Number of relocation entries
    WORD    NumberOfLineNumbers;   // Number of line-number entries
    DWORD   Characteristics;      // Characteristics flags
} SECTIONTABLES, *PSECTIONTABLES;
```

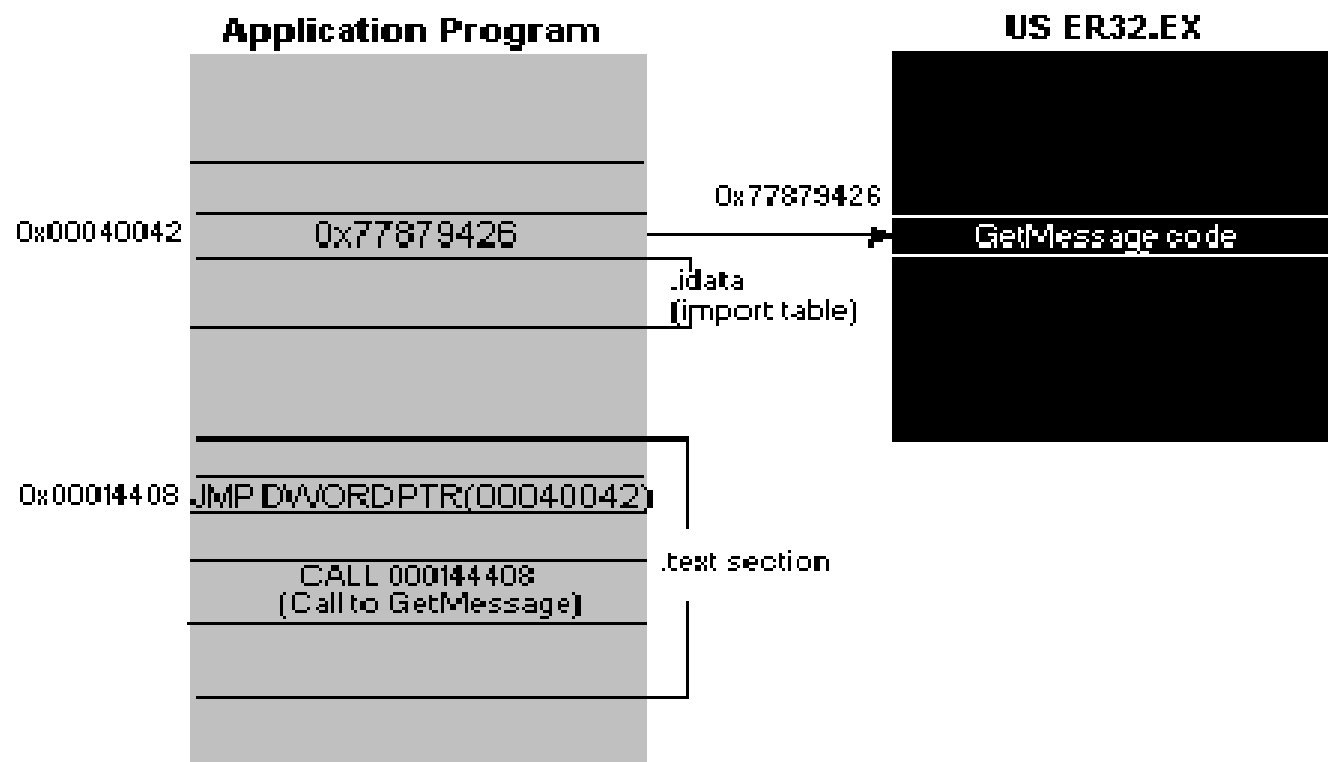
# Format PE

- Symbole PE
  - Data\_Directory[1] – importowane katalogi
    - .idata section
  - Importowane encje z opisanych DDL'ek
    - DLL Name
    - RVA of Import Lookup Table
    - RVA of Import Address Table

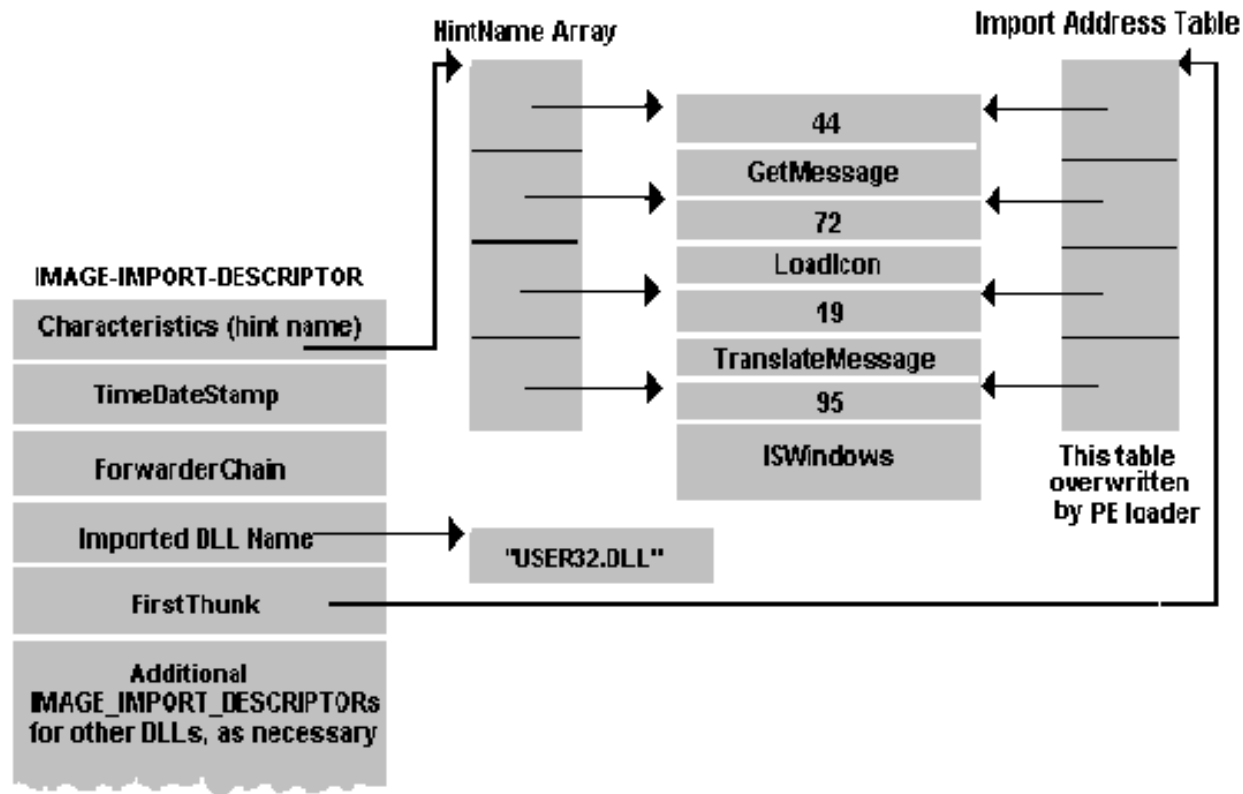


```
typedef struct _IMAGE_IMPORT_DESCRIPTOR {
    union {
        DWORD Characteristics;
        PIMAGE_THUNK_DATA OriginalFirstThunk;
    } DUMMYUNIONNAME;
    DWORD TimeDateStamp;
    DWORD ForwarderChain;
    DWORD Name;
    PIMAGE_THUNK_DATA FirstThunk;
} IMAGE_IMPORT_DESCRIPTOR, *PIMAGE_IMPORT_DESCRIPTOR;
```

# Wywołanie funkcji z innej biblioteki



# Tablica importu



# Tablica eksportu

